Grado en Ingeniería Informática
Sistemas Operativos II

Práctica 2: Sincronización de procesos con semáforos

Objetivo

Conocer el funcionamiento de los mecanismos de solución al problema de las carreras críticas, en particular, los semáforos. Desarrollar habilidades en su manejo.

Desarrollo

1. Estudia el problema del **productor-consumidor** visto en clase usando **procesos y espera activa** para dormir y despertar a los procesos**.**

- Comprueba que si el productor solamente escribe en el `buffer` y el consumidor lee en él utilizando la variable "`cuenta`" pueden presentarse *carreras críticas*. El tamaño del `buffer` debe ser `N=10`, definido como una constante del código.
    - Para comprobarlo puedes aumentar la posibilidad de que ocurran carreras críticas haciendo **que la región crítica dure más tiempo**, por ejemplo, con llamadas a `sleep` de forma que el productor vaya el doble de rápido que el consumidor y viceversa.
- Constata la ocurrencia de carreras críticas.
    - Añade al código **salidas que muestren detalladamente** como van evolucionando tanto el productor como el consumidor.
- Dado que el consumidor y el productor serán dos **procesos diferentes** (*no uses threads*), es necesario un mecanismo para definir zonas de memoria donde almacenar **variables compartidas**:
    - Para definir en cada uno de esos procesos un espacio de memoria compartida con otros procesos, se puede mapear una zona de memoria en todos los procesos con la función **`mmap`**.
    - **Implementa** el productor y el consumidor creando **dos procesos nuevos**, uno para el productor y otro para el consumidor. Pueden ser dos programas diferentes o uno solo.

2. Usando como base el código anterior, programa la solución vista en clase usando semáforos teniendo en cuenta lo siguiente:

- El `buffer` debe ser **tipo `int`** y funcionar como una cola **`LIFO`** (*Last In First Out*).
- El **tamaño** del `buffer` debe ser `N=10` y debe estar inicializado a -1.

- La función **`produce_item()`** debe generar un elemento (un entero).
- La función **`insert_item()`** debe colocar el elemento en el buffer.
- La función **`consume_item()`** debe retirar un entero del buffer y sustituirlo por -1. Además, debe sumar el elemento extraído con el anterior, solo si la posición extraída es mayor que 0.
- Soluciona las carreras críticas utilizando semáforos teniendo en cuenta lo siguiente:
- Los programas deben incluir los siguientes **archivos de cabecera**:
  `<sys/mman.h> <pthread.h> <semaphore.h> <unistd.h> <fcntl.h>`
- Las **variables** semáforo se declaran como punteros a estructuras de tipo *sem_t*:
  `sem_t *vacias;`
- Para **crear un semáforo** usa la función *sem_open*. Un ejemplo de uso es:
  `vacias = sem_open("VACIAS",O_CREAT,0700,N);`
- Uno de los procesos debe crear el semáforo e **inicializarlo** con la función anterior. Los procesos que usen ese semáforo posteriormente deben **abrirlo** pero no inicializarlo. Para ello también usan la función *sem_open* del siguiente modo:
  `vacias = sem_open("VACIAS",0);`
- Para **cerrar los semáforos** y el espacio de memoria compartida, usa las funciones del siguiente ejemplo:
  ```
  sem_close(vacias);
  munmap(buf, N*sizeof(int));
  ```
- Las funciones de **manejo de los semáforos** de POSIX son `sem_wait` y `sem_post`.
- Para no tener que abortar el programa en cada ejecución, no incluyas un lazo externo infinito, pon un **número de iteraciones finito** (por ejemplo 100).
- Añade llamadas a la función `sleep` en los códigos del productor y consumidor **fuera de la región crítica** para forzar situaciones en las que el productor y el consumidor vayan a **diferentes velocidades** de modo que eventualmente el buffer se llegue a llenar o a vaciar según el caso, para que los procesos se bloqueen alguna vez.
  - La versión final **debe incluir** llamadas a `sleep` de valores aleatorios entre `0 y 4` (para generar números aleatorios usa las funciones `rand` y `srand`).
- Al final deben eliminarse los semáforos usando la función **`sem_unlink`**. Si no se usa esta función, los semáforos quedarán activos en el kernel. Una buena práctica es eliminar los semáforos antes de crearlos al principio del programa.
- Para **compilar** se debe incluir la opción de compilación: **`-pthread`**.

3. Resuelve el problema con **`threads`** en lugar de procesos.

Optativo: Generaliza los códigos anteriores con semáforos usando procesos (apartado 2) para un número arbitrario de productores y consumidores.

Formato y fecha de entrega

Hacer un informe breve incluyendo comentarios y las conclusiones obtenidas para los apartados 1, 2 y 3. Sube un único archivo comprimido con zip incluyendo los códigos de los 3 ejercicios con comentarios exhaustivos sobre su funcionamiento.

Fecha de entrega: hasta las 23:59 del día anterior a la sesión de prácticas donde se comience la práctica 3, es decir, G1 miércoles 29 de marzo, G2 lunes 3 de abril, G3 lunes 3 de abril, G4 domingo 2 de abril y G5 jueves 30 de marzo.